



Database Project

Federico Del Pup
Luigi Pascu
Matteo Passador
Stefano Toneguzzo

4 giugno 2026

Indice

1	Analysis of Requirements	1
1.1	Introduction to the Request	1
1.2	Glossary of Terms	2
1.3	Structuration of Requirements	2
1.3.1	General Statements	2
1.3.2	Detail of Requirements for Specific Entities and Relationships . . .	2
1.3.3	Operational Requirements	3
2	Conceptual Design	3
2.1	Publication Management	4
2.2	Management of the Affiliation Entity and Its Specializations	5
2.3	Integration of Components and Final Schema	6
3	Logical Design	7
3.1	Volume Table	7
3.2	Redundancy Analysis	7
3.2.1	Redundant Attributes	8
3.2.2	With Redundancy of the NumeroCitazioni Attribute	8
3.2.3	Without Redundancy of the NumeroCitazioni Attribute	9
3.2.4	Comparative Study	9
3.3	ER schema restructuring	10
3.4	Logical schema	12
4	Physical design (DB implementation)	14
4.1	SQL Table Generation	14
4.2	Database Population	17
4.3	Trigger	18
4.3.1	Trigger for incrementing the number of citations	18
4.3.2	Trigger per il decremento del numero di citazioni	21
4.3.3	Trigger per bloccare l'update delle citazioni	22
4.3.4	Trigger per il controllo dell'affiliazione universitaria nelle tesi	22
4.3.5	Implementazione di trigger esterni alla richiesta	25
5	Query	25
6	Implementazione operazioni principali in SQL	28
7	Autori	29

1 Analysis of Requirements

1.1 Introduction to the Request

The project requires the design of a relational database for managing a bibliography. The database must be able to handle a large set of publications, each characterized by specific attributes and complex relationships with authors, affiliations, publishers, etc. The goal is to create a conceptual model that faithfully represents the domain, followed by a logical

and physical design that optimizes the most frequent operations, such as printing data and inserting new publications. Additionally, the implementation of triggers is required to maintain data integrity and query efficiency, as well as the definition of specific operations and queries for analyzing bibliographic data.

We have evaluated focusing the project on the scope of an academic bibliography within a regional context, in order to have a limited and well-defined scope.

1.2 Glossary of Terms

Term	Description	Relationships
Publication	Bibliographic item	Author, Publication
Journal Article	Type of publication	Author, Publication
Conference Paper	Type of publication	Author, Publication
Book	Type of publication	Author, Publication, Publisher
Thesis	Type of publication	Author, Publication, University
Author	Person who writes a publication	Publication, Affiliation
Affiliation	Group of authors	Author
Publisher	Entity that publishes a book	Book
University	Place of discussion for the thesis	Thesis

1.3 Structuration of Requirements

1.3.1 General Statements

Assume we have collected the following set of requirements for the design of a relational database for managing a bibliography. A bibliography is composed of a set of publications.

1.3.2 Detail of Requirements for Specific Entities and Relationships

- **Publications:** Each publication is characterized by a unique code, a title, a publication year, and one or more authors. A list of bibliographic references (citations) towards other publications in the bibliography is present. Each publication tracks the number of citations received.
- **Journal Articles:** The name of the journal, the volume number, the pages (initial and final) and the total number of pages are recorded.
- **Conference Papers:** The name and location of the conference, the year of execution (independent from the publication year), the pages (initial/final) and the total number of pages are recorded.
- **Books:** Identified by the ISBN code, they include the publisher and the total number of pages.
- **Theses:** The topic, the university of discussion, and the total number of pages are stored.
- **Authors:** For each author, their name, surname, email, website, and one or more affiliations are recorded.

- **Affiliations:** Identified by name, complete address (street, civic number, city, country) and phone number.
- **Publishers:** Identified by name and physical address.
- **Universities:** Identified uniquely by name, including address and phone number.

1.3.3 Operational Requirements

The most frequent operations that the system will need to manage include:

- OP1: Printing of publications, including title, authors, year, and citation count. The estimated frequency is 500 times per day, taking into account that there are periods of intense academic activity (for example, during exam sessions or publication deadlines) when the demand for consulting publications increases significantly.
- OP2: Insertion of a new publication, with the possibility of updating the data and citations. These activities are estimated at around 40 times per day, considering that the publication of new academic works occurs with a certain regularity, but not excessively frequent.
- OP3: Insertion of a new author, with the possibility of associating affiliations. Here we estimate around 10 insertions per day, as the addition of new authors mainly occurs when new researchers enter the system or when collaborations with already existing authors are registered.
- OP4: Modification of an author's affiliation. We have estimated a frequency of 1 time per day for this operation, as academic affiliation changes are relatively rare compared to other operations, but it is still important to manage them correctly to maintain data accuracy.

2 Conceptual Design

The Entity-Relationship schema has been designed to faithfully and comprehensively represent the domain of academic bibliography, taking into account the specific requirements and complex relationships between the involved entities. Below is a detailed description of the main components of the ER schema, with particular attention to the management of entities and relationships more relevant. The schema has been built using a bottom-up technique, starting from individual entities and simpler relationships, then progressively integrating specializations and more complex associations. It is noted that in the ER schema, citations are represented as a recursive relationship between publications.

2.1 Publication Management

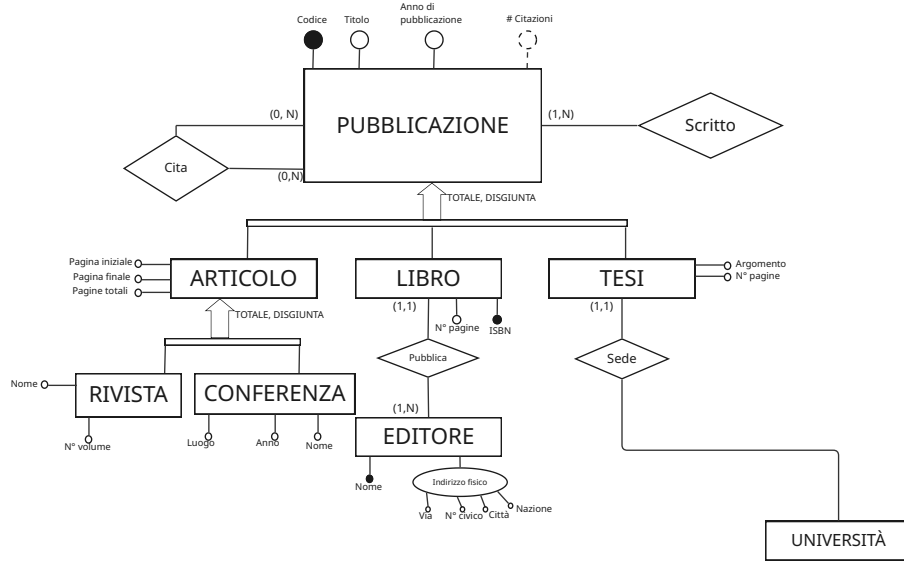


Figura 1: Entità Pubblicazione e specializzazioni

The entity **Publication** has been modeled as a parent entity with three specializations: **Journal Article**, **Book**, and **Thesis**. This choice is motivated by the need to represent clearly and structurally the differences between the various types of publication, which have specific attributes and distinct relationships with other entities (for example, the publisher for books and the university for theses). The specialization has been implemented using a total coverage hierarchy, where each publication must necessarily be classified as one of the three specific types. The entity **Journal Article** is further specialized into **Journal** and **Conference**, as they manage distinct attributes (for example, the volume number for journals and the conference location for conference papers). The specialization is total disjoint, as they cover all articles but do not overlap with each other. The entity **Publisher** is linked to the entity **Book** through a publication relationship, and presents a composite attribute *Address* that includes street, house number, city, and country. The relationship is one to many, as a publisher can publish multiple books. Finally, as written above, citations are managed as a recursive relationship between publications, allowing for an effective modeling of the bibliographic reference dynamics within the system. We assume that the name of the publisher is unique within the domain, given the regional context to which we refer.

2.2 Management of the Affiliation Entity and Its Specializations

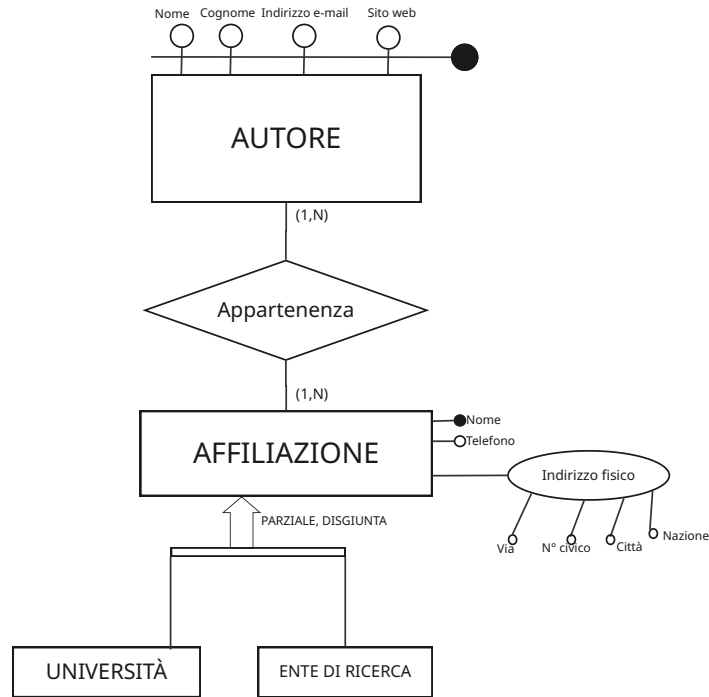


Figura 2: Author, Affiliation and specializations entity

The entity **Affiliation** has been specialized into **University** and **Research Institution** to represent more detailed the different types of affiliations that an author can have. This specialization has been implemented using a partial coverage hierarchy, as there may be affiliations that do not fall into either of the two categories. The choice to keep the child entities separate is motivated by the possibility of later linking the university to theses, while research institutions do not have this relationship. The entity **Author** is linked to the entity **Affiliation** through a membership relationship, which allows modeling the fact that an author can be affiliated with multiple institutions over time. The relationship is many to many, as an author can have multiple affiliations and an affiliation can be associated with multiple authors. It is not required to manage the periods of membership from an author to an affiliation (historical tracking), for example if an author was affiliated with an affiliation in different periods, it is not necessary to keep track of this information.

As stated for the publisher, we assume that the name of the affiliation is unique within the domain, given the regional context to which we refer.

Note that the entity **Research Institution** has been added upon the instructor's suggestion, in order to represent more comprehensively the domain of affiliations, which are not limited exclusively to universities, but also include other types of organizations involved in academic research.

The entity **Author** presents a candidate key, composed by the **superkey**, as it is not specified by the requirements that any combination of attributes could serve as a unique identifier.

2.3 Integration of Components and Final Schema

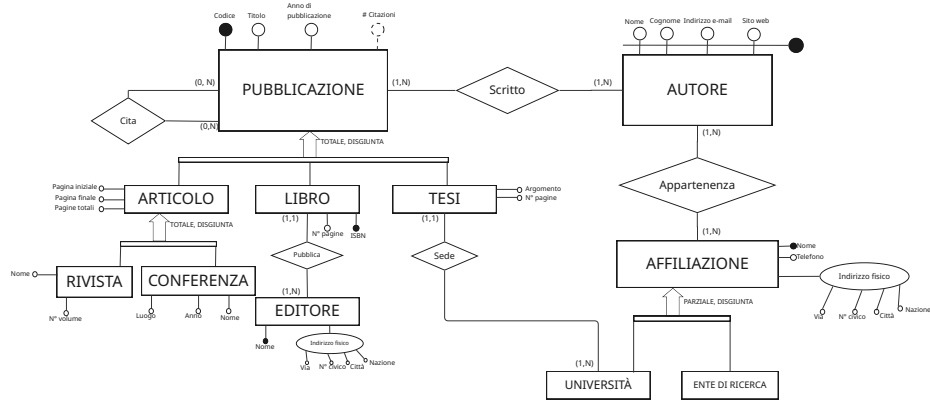


Figura 3: Entity-Relationship Schema

To integrate the produced components, a many-to-many relationship is created between **Author** and **Publication**. Finally, the entity **Thesis** is connected to the entity **University** through a defense relationship, since each thesis is defended at a specific university. This creates a problematic cycle, which is handled in later phases because it could lead to data inconsistencies. For example, an author affiliated with a research institution cannot be associated with a thesis, but if the cycle is not managed, they could be linked to a thesis without having co-authors affiliated with the university where the thesis is defended. Some constraints cannot be resolved through the ER schema. For example, the no self-citation constraint (a publication cannot cite itself) or the acyclicity constraint (if B cites A, A cannot cite B) for citations cannot be managed at the conceptual design level, but it will be necessary to implement triggers at the physical design level to guarantee data integrity in these specific cases.

3 Logical Design

3.1 Volume Table

Concept	Type	Volume
Publication	E	100.000
Journal article	E	20.000
Conference paper	E	15.000
Book	E	10.000
Thesis	E	55.000
Author	E	70.000
Affiliation	E	200
Publisher	E	150
University	E	35
Research institution	E	100
Citation	R	2.000.000 (100.000×20)
Written	R	330.000 ($3,3 \times 100.000$)
Membership	R	140.000 (70.000×2)
Defense	R	55.000
Publishes	R	10.000

The volume table represents the fundamental step from conceptual design to logical design, providing a quantitative estimate of the load the database will have to support. The system manages a total of **100,000 publications**. Note how the sum of the child entities (*Journal article*, *Conference paper*, *Book*, *Thesis*) is exactly 100,000, reflecting the choice of a **total coverage hierarchy**. Theses represent the largest share (55,000), consistent with an academic domain.

With **70,000 authors** and an average of 3.3 authors per publication (relationship **Written**), the system must manage about 330,000 author-publication links. The **Membership** relationship (70.000×2) correctly models academic turnover, assuming that each author is affiliated with two different institutions over time. The small number of **Affiliations (200)**, **Publishers (150)** and **Universities (35)** suggests that these tables will act as "control registries", with a low update rate compared to the transactional tables of publications and citations. In general, these volumes are suitable for a regional context, as hypothesized. Since there can be **Affiliations** that are neither **Universities** nor **Research Institutions**, we estimated 65 affiliations that do not fall into either category, bringing the total to 200. The estimate of 20 citations per publication was made considering that there may be many publications with another degree of internal self-reference within our context.

3.2 Redundancy Analysis

Frequency of the Most Common Operations

Here is an estimate of the frequency of the most common operations the system will have to handle, based on reasonable assumptions consistent with the regional academic domain:

OP1 Publication printing

Frequency: 500 times per day.

OP2 Publication insertion

Insert a publication (40 times per day).

OP3 Insert an author

Frequency: 10 times per day.

OP4 Modify affiliation

Frequency: 1 time per day (estimated frequency).

3.2.1 Redundant Attributes

The only redundant attribute present in the schema is **NumeroCitazioni** in the **Publication** entity. A comparative analysis is made between the two solutions for the attribute, considering the impact of redundancy on system efficiency, particularly for the printing and citation update operations, which are the most frequent. In the redundancy analysis, only operations 1 and 2 are considered because they involve the **Publication** entity and the redundant attribute **NumeroCitazioni**, while operations 3 and 4 do not directly involve this attribute.

3.2.2 With Redundancy of the NumeroCitazioni Attribute

For the cost calculation, it is assumed that an access to an entity or relationship of type "L" (read) has a unit cost of 1, while an access of type "S" (write) has a unit cost of 2 (*ratio 1:2*), reflecting the additional overhead associated with write operations compared to reads. The total cost estimate is made using the higher frequency data for each operation, in order to evaluate the maximum impact of redundancy on system efficiency.

Operation 1

Concept	Construct	Accesses	Type
Publication	Entity	1	L

Cost: $(500 \times 1) \times 1 = 500$ units/day

Operation 2

Concept	Construct	Accesses	Type
Publication	Entity	2	S
Citation	Relationship	1	S
Publication	Entity	1	L
Written	Relationship	1	S

Logical steps:

- 1) Save the publication data
- 2) Save the publication and cited publication pairs

- 3) Find the cited publication
- 4) Increment that publication's number of citations by 1
- 5) Save the publication-author pair

Cost: $(40 \times 4) \times 2 + (40 \times 1) \times 1 = 360$ units/day

3.2.3 Without Redundancy of the NumeroCitazioni Attribute

Operation 1

Concept	Construct	Accesses	Type
Publication	Entity	1	L
Citation	Relationship	20	L

Description: I print the data related to a publication. For each publication, to calculate the number of citations I access the Citation relationship. The average number of accesses per Publication is calculated as: $\frac{\text{Citation volume}}{\text{Publication volume}} = 20$.

Cost: $500 \times (20 + 1) \times 1 = 10.500$ units/day

Operation 2

Concept	Construct	Accesses	Type
Publication	Entity	1	S
Citation	Relationship	1	S
Written	Relationship	1	S

Logical steps:

- 1) Save the publication data
- 2) Save the pairs of publication and cited publication
- 3) Save the publication-author pair

Cost: $40 \times 3 \times 2 = 240$ units/day

3.2.4 Comparative Study

Cost	With redundancy	Without redundancy
OP1	500	10.500
OP2	360	240
Total	860	10.740

Conclusion: Since $860 < 10.740$, it is convenient to keep the derived attribute.

3.3 ER schema restructuring

The following shows the changes made to the ER model to facilitate translation to the relational logical schema, with particular attention to handling specializations and simplifying some complex structures.

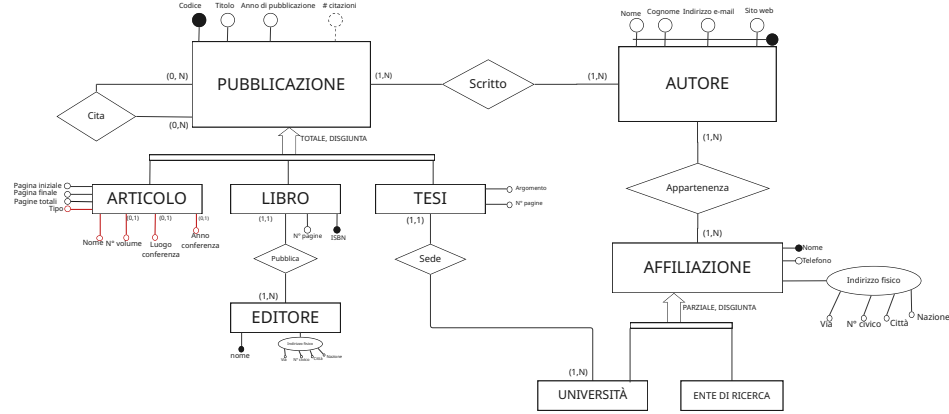


Figura 4: ER schema modified after the first revision

Journal-Conference Specialization: The hierarchy was merged upward (restructuring toward the parent) by removing the child entities and migrating their attributes into the parent entity *Article*. To distinguish occurrences, the discriminating attribute *Type* was introduced, with values constrained to the domain {Journal, Conference}.

The attributes inherited from the child entities were defined as optional (cardinality 0,1), with the exception of the attribute *Name*, which remains mandatory because it is common to both types. It is also specified that the attribute *ConferenceYear* must be semantically distinct from the attribute *PublicationYear*. For example, a conference can be held in a different year than the publication year of the associated article.

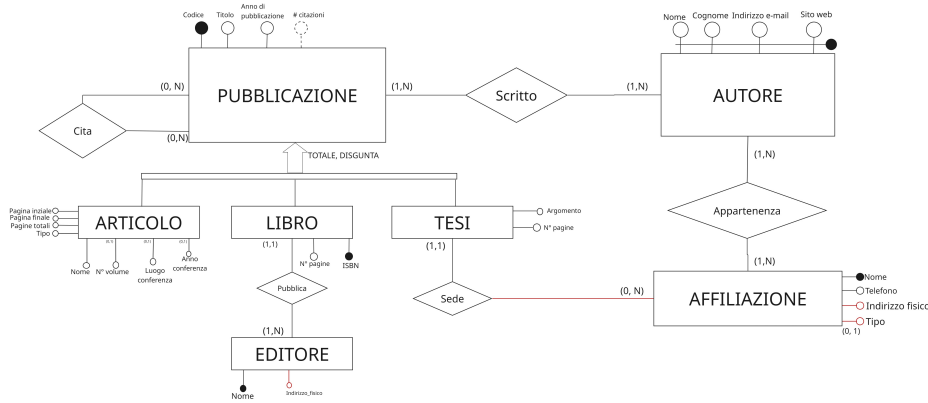


Figura 5: ER schema modified after the second revision

Affiliation Hierarchy Simplification: The specialization between the entity *Affiliation* (parent) and the subentities *University* and *ResearchInstitute* was resolved by merging upward to the parent. This choice is motivated by the absence of relevant specific attributes in the child entities.

To preserve the original semantics, an integrity constraint based on the affiliation type was introduced: association with the entity *Thesis* is valid exclusively for occurrences of type *University*. In terms of cardinality, the relationship between *Thesis* and *Affiliation* is (1, 1), while the participation of the entity *Affiliation* is (0, n), because only the subset related to universities can be associated with a thesis.

Address Attribute Simplification: The composite attribute *PhysicalAddress* was restructured by aggregating its components (such as street, street number, postal code and city). To simplify implementation in the logical model and facilitate insert operations, the individual fields were unified into a single atomic string attribute. This is done because composite attributes do not exist in the relational logical model, so it is necessary to simplify the attribute structure to ensure a more direct translation without loss of information.

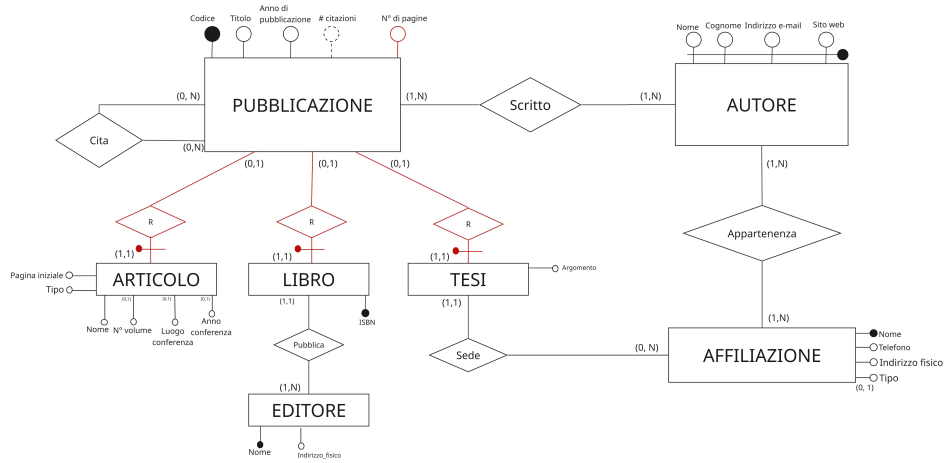


Figura 6: ER schema modified after the third revision

Publication Hierarchy Resolution: The specialization between the parent entity *Publication* and the subentities *Article*, *Book* and *Thesis* was resolved by keeping the entities distinct (multiple entity strategy).

To correctly implement the logical link, explicit relationships between the parent and the children were defined via foreign keys in the child entities referencing the primary key of *Publication*. To ensure total coverage of the specialization, an additional integrity constraint was introduced: each occurrence of the entity *Publication* must necessarily be referenced as a foreign key in at least one of the three child entities.

Page attribute simplifications: Since *TotalPageCount* is a common attribute to all the subentities of the entity **Publication**, it was moved to the parent entity. From there, it was decided to remove the attribute *EndPage* from **Article**, since it can be easily calculated from *StartPage* and *TotalPageCount*.

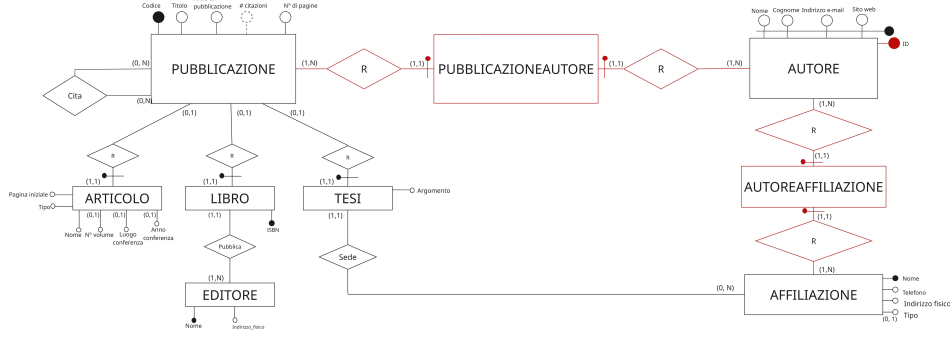


Figura 7: ER schema modified after the fourth revision

"Written" and "Membership" relationship restructuring, key on Author: The relationship **Written** was restructured by introducing an associative entity called **PublicationsAuthors**, which links the entity **Author** with the entity **Publication**. This choice is motivated by the need to correctly represent the many-to-many relationship between authors and publications, allowing effective management of cases where a publication has multiple authors and an author has multiple publications. The same approach was adopted for the *Membership* relationship, with the introduction of the associative entity **AuthorAffiliation** linking the entity **Author** with the entity **Affiliation**. This restructuring makes it possible to model the affiliation dynamics of authors more flexibly, as they can be associated with multiple organizations over time, and to maintain clear traceability of relationships between authors and affiliations.

Finally, it was noted that the entity **Author** has only a superkey, since the requirements do not specify that any attribute can act as a unique identifier. Therefore, a new attribute *ID* was introduced as a candidate key for the entity **Author**, thus ensuring uniqueness.

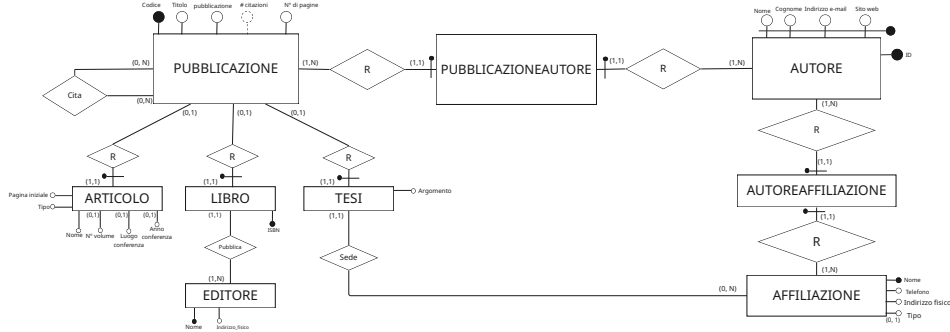


Figura 8: Final ER schema without detailed modifications

It should be noted that the ER model cannot handle the non-self-citation and non-circular-citation constraints. The problematic cycle created by the relationship between *Publication*, *Author* and *Affiliation* is also not manageable. These constraints will be handled at the physical design level, through the implementation of triggers that ensure data consistency during insert and update operations.

3.4 Logical schema

From the final ER model, the following relational logical schema is obtained:
PUBLICATIONS(Code, Title, Year, PageCount, CitationCount)

ARTICLES(PublicationCode, Name, VolumeNumber, ConferenceLocation, ConferenceYear, StartPage, Type)
 BOOKS(PublicationCode, ISBN, PublisherName)
 THESES(PublicationCode, Topic, Affiliation)
 AUTHORS(ID, Name, Surname, Email, Website)
 AFFILIATIONS(Name, Address, Phone, Type)
 PUBLISHERS(Name, Address)
 CITATIONS(CitingPublication, CitedPublication)
 PUBLICATIONAUTHOR(AuthorID, PublicationCode)
 AUTHORAFFILIATION(AuthorID, AffiliationName)

*primary keys are underlined, while foreign keys are indicated with arrows.

Foreign keys:

ARTICOLI.CodicePubblicazione → PUBBLICAZIONI.Codice
 LIBRI.CodicePubblicazione → PUBBLICAZIONI.Codice
 TESI.CodicePubblicazione → PUBBLICAZIONI.Codice
 CITAZIONI.PubblicazioneCitante → PUBBLICAZIONI.Codice
 CITAZIONI.PubblicazioneCitata → PUBBLICAZIONI.Codice
 LIBRI.NomeEditore → EDITORI.Nome
 TESI.Affiliazione → AFFILIAZIONI.Nome
 PUBBLICAZIONEAUTORE.IDAutore → AUTORI.ID
 PUBBLICAZIONEAUTORE.CodicePubblicazione → PUBBLICAZIONI.Codice
 AUTOREAFFILIAZIONE.IDAutore → AUTORI.ID
 AUTOREAFFILIAZIONE.NomeAffiliazione → AFFILIAZIONI.Nome

Note that primary keys were defined to guarantee the uniqueness of each record, while foreign keys were implemented to maintain referential integrity between tables, ensuring that the logical relationships between data are respected. The only candidate key not chosen as the primary key is ISBN in the **LIBRI** table, because the publication code is already a unique identifier sufficient to distinguish every book in the system. A single primary key was preferred to simplify join operations and reduce model complexity, avoiding the need to manage composite or multiple keys for the same entity.

Finally, the PK of **AUTORE** is an artificial primary key, since it is much more intuitive and faster than managing a composite key consisting of multiple attributes (e.g. name, surname, email).

Constraints not managed by the logical schema

The constraints not managed by the logical schema are:

- Non self-citation: a publication cannot cite itself.
- Non circular citation: if publication A cites publication B, then publication B cannot cite publication A.
- Problematic cycle between Publication, Author and Affiliation: an author affiliated with a research institute cannot be associated with a thesis, but if the cycle is not managed they could be associated with a thesis without having co-authors affiliated with the university where the thesis is hosted.
- Total coverage constraint for the Publication-Article/Book/Thesis specialization: every publication must necessarily be classified as one of the three specific types.

- Integrity constraint for the Affiliation-University/Research Institute specialization: association with the Thesis entity is valid exclusively for occurrences of type University.
- Domain constraint for the Type attribute in Article: values must be constrained to the domain Rivista, Conferenza.
- Integrity constraint for Article attributes: the attributes NumeroVolume, LuogoConferenza and AnnoConferenza must be filled only if Type is Conferenza, while the attribute PaginaIniziale must be filled only if Type is Rivista.

The most interesting constraints will be implemented at the physical design level through triggers.

4 Physical design (DB implementation)

The DB was implemented using the PostgreSQL relational database management system, chosen for its robustness, scalability and advanced support for the functionalities required by the project. The physical design followed best practices for performance optimization, data integrity management and maintainability. The following sections describe the main implementation phases, including SQL table generation, database population with sample data and trigger implementation to ensure data consistency.

4.1 SQL Table Generation

Below is the SQL syntax used to create the database tables, in Data Definition Language (DDL). The table definitions include the specification of primary keys, foreign keys, and integrity constraints to ensure data consistency and compliance with the logical relationships between entities.

```
-- Tabella Affiliazioni
CREATE TABLE affiliazioni
(
    nome varchar(100) NOT NULL ,
    telefono varchar(100) NOT NULL ,
    indirizzo varchar(100) NOT NULL ,
    tipo varchar(20),
    CONSTRAINT affiliazione_pkey PRIMARY KEY ( nome )
);

-- Tabella Pubblicazioni
CREATE TABLE pubblicazioni
(
    codice SERIAL NOT NULL ,
    titolo varchar(200),
    annopubblicazione integer NOT NULL ,
    ncitazioni integer CHECK( ncitazioni >= 0) DEFAULT 0 ,
    npagine integer CHECK( npagine > 0) NOT NULL ,
    CONSTRAINT pubblicazione_pkey PRIMARY KEY (codice)
);
```

```

-- Tabella Articoli
CREATE TABLE articoli(
    CodicePubblicazione integer NOT NULL,
    tipo varchar(50) NOT NULL CHECK(tipo IN ('Rivista', 'Conferenza'
        ↳ ')),
    nome varchar(255) NOT NULL,
    Npaginainiziale integer,
    Nvolume integer,
    luogoConferenza varchar(100),
    annoConferenza integer,
    CONSTRAINT articolo_pkey PRIMARY KEY (CodicePubblicazione),
    CONSTRAINT articolo_codice_pubblicazione_fkey FOREIGN KEY (
        ↳ CodicePubblicazione)
    REFERENCES public.pubblicazioni(codice) MATCH SIMPLE
    ON UPDATE NO ACTION
    ON DELETE CASCADE
);

-- Tabella AutoreAffiliazione
CREATE TABLE autoreaffiliazione
(
    idAutore integer NOT NULL,
    nomeAffiliazione varchar(255) NOT NULL,
    CONSTRAINT autoreaffiliazione_pkey PRIMARY KEY (idautore,
        ↳ nomeaffiliazione),
    CONSTRAINT auto_aff_id_autore_fkey FOREIGN KEY (idautore)
    REFERENCES public.autori(id) MATCH SIMPLE
    ON UPDATE CASCADE
    ON DELETE CASCADE,
    CONSTRAINT auto_aff_nome_affiliazione_fkey FOREIGN KEY(
        ↳ nomeaffiliazione)
    REFERENCES public.affiliazioni(nome) MATCH SIMPLE
    ON UPDATE CASCADE
    ON DELETE CASCADE
);

-- Tabella Autori
CREATE TABLE autori
(
    id SERIAL NOT NULL,
    nome varchar(100) NOT NULL,
    cognome varchar(100) NOT NULL,
    email varchar(100) NOT NULL,
    sitoweb varchar(100) NOT NULL,
    CONSTRAINT autore_pkey PRIMARY KEY(id)
);

-- Tabella Citazioni
CREATE TABLE citazioni
(
    pubblicazioneCitante integer NOT NULL ,

```



```

pubblicazioneCitata integer NOT NULL ,
CONSTRAINT citazione_pkey PRIMARY KEY (pubblicazioneCitante,
    ↳ pubblicazioneCitata),
CONSTRAINT citazione_codice_pubblicazione_citante_fkey FOREIGN
    ↳ KEY (pubblicazioneCitante)
REFERENCES public.pubblicazioni(codice) MATCH SIMPLE
ON UPDATE NO ACTION
ON DELETE CASCADE,
CONSTRAINT citazione_codice_pubblicazione_citata_fkey FOREIGN
    ↳ KEY (pubblicazioneCitata)
REFERENCES public.pubblicazioni(codice) MATCH SIMPLE
ON UPDATE NO ACTION
ON DELETE CASCADE
);
-- Tabella Editori
CREATE TABLE editori
(
    nome varchar(100) NOT NULL,
    indirizzo varchar(255) NOT NULL,
    CONSTRAINT editore_pkey PRIMARY KEY(nome)
);
-- Tabella Libri
CREATE TABLE libri
(
    CodicePubblicazione integer NOT NULL,
    isbn varchar(17) UNIQUE NOT NULL,
    nomeEditore varchar(100) NOT NULL,
    CONSTRAINT libro_pkey PRIMARY KEY (CodicePubblicazione),
    CONSTRAINT libro_codice_pubblicazione_fkey FOREIGN KEY (
        ↳ CodicePubblicazione)
REFERENCES public.pubblicazioni(codice) MATCH SIMPLE
ON UPDATE NO ACTION
ON DELETE CASCADE,
CONSTRAINT libro_nome_editore_fkey FOREIGN KEY (nomeEditore)
REFERENCES public.editori(nome) MATCH SIMPLE
ON UPDATE NO ACTION
ON DELETE NO ACTION
);
-- Tabella PubblicazioneAutore
CREATE TABLE pubblicazioneautore
(
    CodicePubblicazione integer NOT NULL,
    idAutore integer NOT NULL,
    CONSTRAINT pubb_autore_pkey PRIMARY KEY (CodicePubblicazione,
        ↳ idAutore),
    CONSTRAINT pubb_autore_codice_pubblicazione_fkey FOREIGN KEY (
        ↳ CodicePubblicazione)
REFERENCES public.pubblicazioni(codice) MATCH SIMPLE
ON UPDATE NO ACTION
ON DELETE CASCADE,
CONSTRAINT pubb_autore_id_autore_fkey FOREIGN KEY (idAutore)

```

```

REFERENCES public.autori(id) MATCH SIMPLE
ON UPDATE NO ACTION
ON DELETE CASCADE
);

-- Tabella Tesi
CREATE TABLE tesi
(
    CodicePubblicazione integer NOT NULL ,
    argomento text NOT NULL ,
    nomeAffiliazione varchar (255) NOT NULL ,
    CONSTRAINT tesi_pkey PRIMARY KEY ( CodicePubblicazione ) ,
    CONSTRAINT tesi_codice_pubblicazione_fkey FOREIGN KEY (
        ↪ CodicePubblicazione)
REFERENCES public.pubblicazioni(codice) MATCH SIMPLE
ON UPDATE NO ACTION
ON DELETE CASCADE ,
    CONSTRAINT tesi_nome_affiliazione_fkey FOREIGN KEY (
        ↪ nomeAffiliazione)
REFERENCES public.affiliazioni ( nome ) MATCH SIMPLE
ON UPDATE NO ACTION
ON DELETE NO ACTION
);

```

4.2 Database Population

In this phase, the insertion of structured data into our database is carried out. To optimize time, the process was implemented using AI prompts, which allowed the generation of consistent and representative data records for the regional academic domain, drastically reducing manual data entry time and minimizing the risk of syntax errors.

For data extraction and formatting, we used **Gemini 3.1 Pro**, an advanced artificial intelligence language model developed by Google, to generate consistent and representative records for the academic domain. The population of the tables was limited to the Triveneto area (*Veneto, Trentino-Alto Adige, Friuli-Venezia Giulia*). This strategic choice was dictated by a key factor: Triveneto boasts a solid and historic presence of numerous activities, universities, and centers of research excellence. Focusing the scope on this territory made it possible to test the effectiveness of the prompt and the robustness of the database architecture on a dataset consistent with the volumes estimated in the volume table.

Here is an example of a table resulting from the database population:

Tabella 1: Affiliazione

Affiliation			
Name	Phone	Address	Type
Università degli Studi di Udine	0432-556111	Via Palladio 8, Udine	Università
Università degli Studi di Trieste	040-5587111	Piazzale Europa 1, Trieste	Università
SISSA	040-3787111	Via Bonomea 265, Trieste	Università
Università degli Studi di Padova	049-8275111	Via 8 Febbraio 2, Padova	Università
Università Ca' Foscari Venezia	041-2345111	Dorsoduro 3246, Venezia	Università
Area Science Park	040-3755111	Padriciano 99, Trieste	Ente di ricerca
OGS	040-21401	Borgo Grotta Gigante 42/C, Sgonico	Ente di ricerca
CRO Aviano	0434-659111	Via Franco Gallini 2, Aviano	Ente di ricerca
Università degli Studi di Trento	0471-011000	Via Calepina 14, Trento	Università
Libera Università di Bolzano	0471-011000	Piazza Università 1, Bolzano	Università
Fondazione Bruno Kessler	0461-314200	Via Santa Croce 77, Trento	Ente di ricerca

4.3 Trigger

At the instructor's request, we decided to implement the most interesting and relevant triggers for our domain, particularly those related to the management of citations.

Logical properties of citation constraints

Non Autocitazione: Una pubblicazione non può citare sé stessa. Questo è il vincolo più elementare e serve a prevenire l'inserimento di dati logicamente inconsistenti, garantendo che il conteggio delle citazioni rifletta solo i riferimenti esterni.

Non Citazione Circolare: Una pubblicazione A non può citare una pubblicazione B se B cita già A, direttamente o indirettamente. Questo vincolo impedisce la formazione di cicli di citazione che potrebbero distorcere il conteggio delle citazioni e complicare l'analisi bibliometrica.

Nota sul Vincolo Temporale: Una pubblicazione non può citare un'altra pubblicazione con un anno di pubblicazione successivo. Pur riconoscendo l'esistenza di casi come i preprint nel mondo reale, questo vincolo è assunto logicamente come rilevante ed è implementato nel database.

4.3.1 Trigger for incrementing the number of citations

The trigger `inc_n_citazioni` is designed to handle the increment of the citation count for a publication whenever a new citation involving it is inserted. The trigger performs the following operations: 1. It checks that the citing publication is not the same as the cited publication, thus preventing self-citation. 2. It ensures that the citing publication does not cite a publication with a later publication year, guaranteeing temporal consistency of citations. 3. If the checks are passed, the trigger increments the citation count of the cited publication, thus updating the derived attribute **NumeroCitazioni** accurately and promptly.

```
-- 1. TRIGGER PER L'INSERIMENTO (Incremento e Controlli)
CREATE OR REPLACE FUNCTION inc_n_citazioni()
```

```

RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    v_anno_citante integer;
    v_anno_citata integer;
BEGIN
    -- Controllo auto-citazione immediato
    IF NEW.pubblicazioneCitante = NEW.pubblicazioneCitata THEN
        RAISE EXCEPTION 'Una pubblicazione non si può citare da
        ↳ sola';
    END IF;

    -- Recupero gli anni di pubblicazione (usando i nomi
    ↳ corretti dello schema)
    SELECT annopubblicazione INTO v_anno_citante
    FROM public.pubblicazioni
    WHERE codice = NEW.PubblicazioneCitante;

    SELECT annopubblicazione INTO v_anno_citata
    FROM public.pubblicazioni
    WHERE codice = NEW.PubblicazioneCitata;

    -- Controllo temporale
    IF v_anno_citante < v_anno_citata THEN
        RAISE EXCEPTION 'Una pubblicazione può citare solo
        ↳ pubblicazioni contemporanee o più vecchie';
    END IF;

    --Se A cita B, B non può citare A
    IF v_anno_citante = v_anno_citata THEN
        IF EXISTS( SELECT * FROM CITAZIONI
        ↳ WHERE pubblicazionecitata = NEW.
        ↳ PubblicazioneCitante AND
        ↳ pubblicazionecitante = NEW.
        ↳ PubblicazioneCitata) THEN
            RAISE EXCEPTION 'Creazione di una
            ↳ citazione ciclica';
        END IF;
    END IF;

    --Aggiornamento
    UPDATE public.pubblicazioni
    SET ncitazioni = ncitazioni + 1
    WHERE codice = NEW.PubblicazioneCitata;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_ins_CIT
BEFORE INSERT ON public.citazioni
FOR EACH ROW EXECUTE FUNCTION inc_n_citazioni();

```

Test

Ecco degli esempi di casi di test per la validazione dei vincoli di integrità sulle citazioni, che coprono scenari di inserimento valido, tentativi di autocitazione e violazioni della coerenza temporale. Ad esempio, per il caso TC-01 (inserimento valido di una citazione verso una pubblicazione antecedente), il codice SQL potrebbe essere:

```
-- TC-01: Inserimento valido di una citazione
INSERT INTO citazioni (pubblicazioneCitante, pubblicazioneCitata)
    ↪ VALUES (1001, 1002);
```

Di seguito si propone la seguente test suite:

Tabella 2: Test cases for the validation of citation integrity constraints.

Test ID	Scenario / Description	Input Data	Expected Result
TC-01: Caso Valido	Inserimento valido di una citazione verso una pubblicazione antecedente.	Pubbl. Citante: 1001 (2024) Pubbl. Citata: 1002 (2022)	Successo: Transazione eseguita e record inserito.
TC-02: Caso di Autocitazione	Tentativo di autocitazione da parte di una pubblicazione.	Pubbl. Citante: 1001 (2024) Pubbl. Citata: 1001 (2024)	Errore: "Una pubblicazione non si può citare da sola"
TC-03: Caso di Citazione Circolare	Tentativo di citazione circolare tra due pubblicazioni.	Pubbl. Citante: 1001 (2024) Pubbl. Citata: 1002 (2024) Pubbl. Citante: 1002 (2024) Pubbl. Citata: 1001 (2024)	Errore: "Creazione di una citazione ciclica"
TC-04: Violazione della Coerenza Temporale	Violazione della coerenza temporale (citazione verso il futuro).	Pubbl. Citante: 1001 (2024) Pubbl. Citata: 1003 (2026)	Errore: "Una pubblicazione può citare solo pubblicazioni contemporanee o più vecchie"

Nota sulle citazioni circolari fra pubblicazioni dello stesso anno

Il vincolo di non ciclicità presenta una zona grigia specifica al di fuori di questo trigger: le pubblicazioni dello stesso anno. Il controllo temporale impone che una pubblicazione citi solo opere con anno minore o uguale al proprio. Questo di per sé eliminerebbe già tutti i cicli temporali, perchè richiederebbe che ogni pubblicazione citi la successiva.

Tuttavia, se per esempio, tre pubblicazioni sono state pubblicate nello stesso anno, potrebbero citarsi reciprocamente senza violare il vincolo temporale. In questo caso, il vincolo di non citazione circolare non è applicabile, poichè il confronto fra citazioni è sempre soddisfatto. Di seguito un'illustrazione

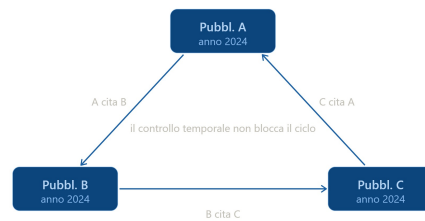


Figura 9: Esempio di citazioni circolari fra pubblicazioni dello stesso anno.

Per gestire questo caso, sono state valutate tre strategie opposte:

Strategia 1: Non gestire tutti i possibili cicli intra-anno

Questa è la strategia più conservativa, che accetta la possibilità di citazioni circolari all'interno dello stesso anno. Si può decidere di gestire velocemente solo i cicli con due componenti. Questa strategia si giustifica perché evita la sovraccaricazione della base di dati con un'ulteriore operazione complessa dal punto di vista del costo temporale. Questa scelta privilegia le prestazioni, di conseguenza è quella che è stata selezionata come opzione migliore.

Strategia 2: Sostituire l'anno con una data completa

Cambiando l'attributo *AnnoPubblicazione* con una data completa (es. *DataPubblicazione*), si potrebbe implementare un controllo più preciso che impedisce le citazioni circolari anche fra pubblicazioni dello stesso anno. Tuttavia, questa strategia richiede una modifica non richiesta dai requisiti del progetto, comporta un aumento della complessità del modello e potrebbe non essere giustificata se i dati disponibili non includono informazioni più granulari sulla data di pubblicazione.

Strategia 3: Implementare un controllo di ciclicità più complesso

In alternativa, per un controllo rigoroso a livello logico, si potrebbe implementare un algoritmo di rilevamento dei cicli. Un algoritmo pensato per questo scopo è quello basato sulla visita in profondità (DFS) del grafo, mantenendo come vertici le pubblicazioni e come archi le citazioni. Per evitare che questa visita diventi troppo onerosa, si è pensato che andrebbe limitata ad un certo numero di livelli (es. 3-4 livelli) per evitare un'esplorazione troppo ampia del grafo. L'analisi di questa strategia è che avendo un alto numero di citazioni per pubblicazione, la verifica del grafo ad ogni inserimento risulta sempre troppo onerosa (complessità dell'algoritmo DFS è $O(V+E)$, dove V è il numero di pubblicazioni e E è il numero di citazioni). Si conclude che il tradeoff tra coerenza logica e prestazioni è troppo sbilanciato con questa strategia.

4.3.2 Trigger per il decremento del numero di citazioni

A seguito di una cancellazione di una citazione, è necessario decrementare il numero di citazioni della pubblicazione citata.

```

-- 2. TRIGGER PER LA CANCELLAZIONE (Decremento)
CREATE OR REPLACE FUNCTION dec_n_citazioni()
  RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
  -- Aggiornamento diretto senza bisogno di SELECT preventiva
  UPDATE public.pubblicazioni

```

```

        SET ncitazioni = ncitazioni - 1
        WHERE codice = OLD.PubblicazioneCitata;

    RETURN OLD;
END;
$$;

CREATE TRIGGER trg_rim_CIT
BEFORE DELETE ON public.citazioni
FOR EACH ROW EXECUTE FUNCTION dec_n_citazioni();

```

4.3.3 Trigger per bloccare l'update delle citazioni

Il trigger non permette l'aggiornamento delle citazioni. O si inserisce una nuova citazione o si elimina quella vecchia, ma non si può modificare una citazione esistente. Questo è un vincolo logico che serve a mantenere la coerenza dei dati e a prevenire modifiche che potrebbero alterare in modo non tracciabile il conteggio delle citazioni.

```

-- 3. TRIGGER PER BLOCCARE L'UPDATE
CREATE OR REPLACE FUNCTION updt_citazioni()
    RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    RAISE EXCEPTION 'Una citazione non può essere aggiornata.
    ↪ Elimina la vecchia e inseriscine una nuova.';
END;
$$;

CREATE TRIGGER trg_upd_CIT
BEFORE UPDATE ON public.citazioni
FOR EACH ROW EXECUTE FUNCTION updt_citazioni();

```

4.3.4 Trigger per il controllo dell'affiliazione universitaria nelle tesi

Il trigger `trg_TESI` è progettato per garantire che ogni tesi inserita nel database sia associata a un'affiliazione di tipo "Università" e che questa affiliazione sia effettivamente collegata ad almeno uno degli autori della tesi. Questo controllo è fondamentale per mantenere l'integrità dei dati e assicurare che le tesi siano correttamente categorizzate all'interno del contesto accademico.

Il trigger esegue le seguenti verifiche:

- 1) Controlla che il codice della pubblicazione associato alla tesi non sia già utilizzato in altre pubblicazioni (articoli o libri), prevenendo così duplicazioni.
- 2) Verifica che l'affiliazione selezionata per la tesi sia effettivamente un'università presente nel database.
- 3) Assicura che l'affiliazione universitaria sia collegata ad almeno uno degli autori della tesi, garantendo così la coerenza tra l'affiliazione e i contributori della pubblicazione.

```

CREATE OR REPLACE FUNCTION check_uni()
RETURNS TRIGGER
LANGUAGE plpgsql AS
$$
BEGIN
IF EXISTS(SELECT CodicePubblicazione FROM ARTICOLI WHERE NEW.
    ↳ CodicePubblicazione = CodicePubblicazione)
OR EXISTS(SELECT CodicePubblicazione FROM LIBRI WHERE NEW.
    ↳ CodicePubblicazione = CodicePubblicazione) THEN
    RAISE EXCEPTION 'Il codice della pubblicazione è
        ↳ attualmente in uso';
    RETURN NULL;
END IF;

IF NOT EXISTS(SELECT nome FROM AFFILIAZIONI WHERE NEW.
    ↳ nomeAffiliazione = nome AND tipo = 'Università') THEN
    RAISE EXCEPTION 'Università inserita non corretta o non
        ↳ presente nel db';
END IF;

IF NEW.nomeAffiliazione NOT IN(SELECT AA.nomeAffiliazione
                                FROM PUBBLICAZIONEAUTORE PA,
                                ↳ AUTOREAFFILIAZIONE AA
                                WHERE PA.CodicePubblicazione =
                                ↳ NEW.CodicePubblicazione
                                ↳ AND AA.idAutore = PA.
                                ↳ idAutore) THEN
    RAISE EXCEPTION 'Università inserita non è affiliata
        ↳ neanche ad un autore';
END IF;

RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_TESI
BEFORE INSERT OR UPDATE ON TESI
FOR EACH ROW EXECUTE FUNCTION check_uni();

```

Test

Ecco alcuni casi di test per validare il corretto funzionamento del trigger `trg_TESI`, che verifica l'associazione corretta tra tesi, affiliazione universitaria e autori. La test suite include scenari di inserimento valido, tentativi di inserimento con affiliazioni non universitarie e casi in cui l'affiliazione universitaria non è collegata a nessun autore della tesi.

Tabella 3: Test cases for the validation of integrity and affiliation constraints of Theses.

Test ID	Scenario / Description	Input Data	Expected Result
TC-01: Affiliazione non universitaria	Tentativo di inserire una tesi indicando un'affiliazione censita nel sistema, ma che non è un'istituzione universitaria (bensì un Ente di Ricerca).	Cod. Pubblicazione: 4001 Argomento: 'Bioinformatica applicata' Affiliazione inserita: 'CRO Aviano' Affiliazione autori: 'CRO Aviano'	Errore: "Università inserita non corretta o non presente nel db"
TC-02: Discordanza di affiliazioni	Tentativo di inserire una tesi indicando un'Università valida nel sistema, ma alla quale nessun autore della pubblicazione risulta affiliato.	Cod. Pubblicazione: 4001 Argomento: 'Bioinformatica applicata' Affiliazione inserita: 'Università degli Studi di Padova', 'SISSA' Affiliazione autori: 'Università degli Studi di Udine'	Errore: "Università inserita non è affiliata neanche ad un autore"
TC-03: Caso Valido	Inserimento valido di una tesi in cui l'università indicata è corretta e coincide con l'affiliazione di almeno un autore.	Cod. Pubblicazione: 4001 Argomento: 'Bioinformatica applicata' Affiliazione inserita: 'Università degli Studi di Udine' Affiliazione autori: 'Università degli Studi di Udine'	Successo: Transazione eseguita e record inserito nella tabella <code>tesi</code> .

4.3.5 Implementazione di trigger esterni alla richiesta

Per garantire l'integrità dei dati all'interno del database, sono stati implementati altri trigger esclusi dalla richiesta:

- **Vincoli di Esclusività (Generalizzazione):** Nella gestione delle entità **Articolo** e **Libro**, è stato predisposto un trigger che verifica l'unicità del codice di pubblicazione, accertandosi che un record inserito in una tabella non sia già presente nell'altra. Analogamente, per la tabella **Articolo**, un apposito trigger controlla la mutua esclusività degli attributi in base alla tipologia: se l'articolo è di tipo **Conferenza**, viene vincolato l'inserimento dei soli attributi specifici per le conferenze, e viceversa per la tipologia **Rivista**.
- **Integrità Referenziale Complessa:** Nelle tabelle di associazione **PubblicazioneAutore** e **AutoreAffiliazione**, sono stati inseriti dei trigger di controllo per verificare la corretta esistenza e corrispondenza delle rispettive chiavi esterne prima di consentire qualsiasi operazione di inserimento o modifica.

5 Query

Di seguito vengono presentate le query SQL per rispondere alle domande specifiche richieste

Q1 Autore con più pubblicazioni per ogni affiliazione

Stampare il nome dell'autore che detiene il maggior numero di pubblicazioni per ogni singola affiliazione.

Q2 Co-autoria esclusiva

Stampare le coppie di autori che hanno collaborato sempre e solo tra di loro (ovvero, ogni loro pubblicazione è stata fatta in coppia).

Q3 Autori fedeli all'affiliazione

Stampare gli autori che non hanno mai prodotto pubblicazioni con co-autori appartenenti a un'affiliazione diversa dalla propria.

Q4 Analisi dell'impatto a breve termine

Trovare gli autori che hanno pubblicato esclusivamente articoli i quali, a distanza di 2 anni dalla data di pubblicazione, hanno accumulato almeno 5 citazioni.

Q1: Autore con più pubblicazioni per ogni affiliazione

Stampare il nome dell'autore che detiene il maggior numero di pubblicazioni per ogni singola affiliazione.

```
SELECT a.Nome, a.Cognome, aa.NomeAffiliazione
FROM AUTORI a
JOIN PUBBLICAZIONEAUTORE pa ON a.ID = pa.IDAutore
JOIN AUTOREAFFILIAZIONE aa ON a.ID = aa.IDAutore
GROUP BY a.ID, a.Nome, a.Cognome, aa.NomeAffiliazione
HAVING COUNT(pa.CodicePubblicazione) = (
    SELECT MAX(cnt)
```

```

FROM (
    SELECT COUNT(pa2.CodicePubblicazione) AS cnt
    FROM AUTORI a2
    JOIN PUBBLICAZIONEAUTORE pa2 ON a2.ID = pa2.IDAutore
    JOIN AUTOREAFFILIAZIONE aa2 ON a2.ID = aa2.IDAutore
    WHERE aa2.NomeAffiliazione = aa.NomeAffiliazione
    GROUP BY a2.ID
)
);

```

Dal punto di vista del codice, per calcolare il massimo numero di pubblicazioni per ogni istituto, è necessario unire le tabelle relative agli autori, alle pubblicazioni e alle affiliazioni. Si raggruppano quindi i risultati per autore e affiliazione. Per filtrare correttamente il "vincitore" di ogni ente, si utilizza una clausola HAVING che confronta il conteggio delle pubblicazioni del singolo autore con un valore MAX(cnt) ottenuto da una subquery calcolata per la stessa affiliazione.

Q2: Coppie che hanno pubblicato sempre e solo insieme

Stampare le coppie di autori che hanno collaborato sempre e solo tra di loro (ovvero, ogni loro pubblicazione è stata fatta in coppia).

```

SELECT pa1.idAutore, pa2.idAutore
FROM pubblicazioneautore pa1
JOIN pubblicazioneautore pa2
    ON pa1.CodicePubblicazione = pa2.CodicePubblicazione
    AND pa1.idAutore < pa2.idAutore
GROUP BY pa1.idAutore, pa2.idAutore
HAVING COUNT(DISTINCT pa1.CodicePubblicazione) = (
    SELECT COUNT(*)
    FROM pubblicazioneautore pa
    WHERE pa.idAutore = pa1.idAutore
)
AND COUNT(DISTINCT pa2.CodicePubblicazione) = (
    SELECT COUNT(*)
    FROM pubblicazioneautore pa
    WHERE pa.idAutore = pa2.idAutore
);

```

Per implementare questa operazione, che cerca le coppie di autori che hanno collaborato sempre e solo tra di loro, si è deciso di utilizzare una join sulla tabella di associazione PUBB_AUTORE. Per evitare di analizzare due volte la stessa coppia o contare permutazioni speculari, si applica la condizione `pa1.id_autore < pa2.id_autore`. Successivamente, la condizione di esclusività viene garantita controllando tramite HAVING che il numero di pubblicazioni fatte insieme coincida con il totale assoluto delle pubblicazioni del primo autore e, simultaneamente, del secondo autore.

Q3: Autori che non hanno mai pubblicato con colleghi di altre affiliazioni

Stampare gli autori che non hanno mai prodotto pubblicazioni con co-autori appartenenti a un'affiliazione diversa dalla propria.

```
SELECT a.id, a.nome, a.cognome
FROM autori a
WHERE NOT EXISTS (
    SELECT 1
    FROM pubblicazioneautore pa1
    JOIN pubblicazioneautore pa2
        ON pa1.CodicePubblicazione = pa2.CodicePubblicazione
        AND pa1.idAutore <> pa2.idAutore
    JOIN autoreaffiliazione aa1
        ON aa1.idAutore = pa1.idAutore
    JOIN autoreaffiliazione aa2
        ON aa2.idAutore = pa2.idAutore
    WHERE pa1.idAutore = a.id
        AND aa1.nomeAffiliazione <> aa2.nomeAffiliazione
);
```

L'implementazione di questa ricerca si basa su un approccio logico per sottrazione. Si utilizza la clausola NOT EXISTS per scartare tutti quegli autori che possiedono almeno una pubblicazione co-firmata con un ricercatore di un'altra struttura. Nello specifico, la subquery ricerca casi in cui, a parità di codice_pubblicazione, due co-autori risultino avere un'affiliazione differente (aa1.nome_affiliazione <> aa2.nome_affiliazione).

Q4: Autori con solo articoli di successo (min. 5 citazioni a 2 anni)

Trovare gli autori che hanno pubblicato esclusivamente articoli i quali, a distanza di 2 anni dalla data di pubblicazione, hanno accumulato almeno 5 citazioni.

```
SELECT a.ID, a.Nome, a.Cognome
FROM AUTORI a
WHERE EXISTS (
    SELECT 1 FROM PUBBLICAZIONEAUTORE pa WHERE pa.IDAutore = a.ID
)
AND NOT EXISTS (
    SELECT 1
    FROM PUBBLICAZIONEAUTORE pa
    JOIN PUBBLICAZIONI p ON pa.CodicePubblicazione = p.Codice
    WHERE pa.IDAutore = a.ID
        AND (
            SELECT COUNT(*)
            FROM CITAZIONI c
            JOIN PUBBLICAZIONI pc ON c.PubblicazioneCitante = pc.
                ↳ Codice
            WHERE c.PubblicazioneCitata = p.Codice
                AND (pc.AnnoPubblicazione - p.AnnoPubblicazione) <= 2
        ) < 5
);
```

);

Per imporre il vincolo temporale richiesto (conteggio nei primi 2 anni), le citazioni vengono calcolate dinamicamente interrogando la tabella CITAZIONE e incrociando l'anno di pubblicazione. La logica richiede che l'autore abbia pubblicato almeno un articolo (Condizione A con EXISTS) e che non abbia all'attivo alcuna pubblicazione con meno di 5 citazioni nel biennio (Condizione B con NOT EXISTS).

Nota sull'inefficacia dell'attributo derivato nella query Q4: Nonostante la presenza dell'attributo derivato NumeroCitazioni nell'entità PUBBLICAZIONE — introdotto per ottimizzare la stampa dei dati generali (OP1) — esso risulta inutilizzabile ai fini della query OP8 (*Analisi dell'impatto a breve termine*).

Il limite risiede nella natura stessa dell'attributo ridondante:

- **Staticità del dato:** L'attributo memorizza esclusivamente il conteggio totale e cumulativo delle citazioni ricevute fino all'istante attuale.
- **Vincolo Temporale Dinamico:** La specifica della Q4 richiede il filtraggio delle sole citazioni avvenute entro un arco temporale di **due anni** dalla data di rilascio della pubblicazione.

Di conseguenza, per soddisfare il requisito, è necessario ignorare il valore pre-calcolato ed effettuare un calcolo dinamico accedendo direttamente alla relazione CITAZIONI. Questo processo permette di confrontare gli attributi Anno della pubblicazione citante e di quella citata, operazione indispensabile per isolare il sottoinsieme di riferimenti bibliografici pertinenti all'analisi richiesta.

6 Implementazione operazioni principali in SQL

Di seguito vengono presentati i codici SQL per implementare le operazioni principali richieste:

```
-- OPERAZIONE 1
SELECT * FROM PUBBLICAZIONI;

-- OPERAZIONE 2
INSERT INTO PUBBLICAZIONI (titolo, annopubblicazione, npagine)
  ➔ VALUES ('P vs NP', 2021, 100);

-- OPERAZIONE 3
INSERT INTO AUTORI (Nome, Cognome, Email, SitoWeb) VALUES ('Pinco',
  ➔ 'Pallo', 'pp@tim.it', 'www.pinco.it');

-- OPERAZIONE 4
UPDATE AFFILIAZIONI SET indirizzo = 'Piazza Europa, Trento' WHERE
  ➔ indirizzo = 'Piazzale Europa, Trento';
```

7 Autori

Tabella 4: Autori del progetto.

Nome	Cognome	E-mail	Matricola
Stefano	Toneguzzo	stefano.toneguzzo@spes.uniud.it	168579
Luigi	Pascu	luigi.pascu@spes.uniud.it	166851
Matteo	Passador	matteo.passador@spes.uniud.it	168215
Federico	Del Pup	federico.delpup@spes.uniud.it	167087